



Advanced Component

Aug - 2023

OPEN MIND - OPEN DOORS



Table of content

Dialog	1
--------	---

Setting Screen	3
----------------	---

ViewPager2	7
------------	---

NavigationView	12
----------------	----

NavigationDrawer	16
------------------	----

RecyclerView, DiffUtil, Adapter	
---------------------------------	--

Table of content

Dialog	1
Setting Screen	3
ViewPager2	7
NavigationView	12
NavigationDrawer	16
RecyclerView, DiffUtil, Adapter	

Dialog

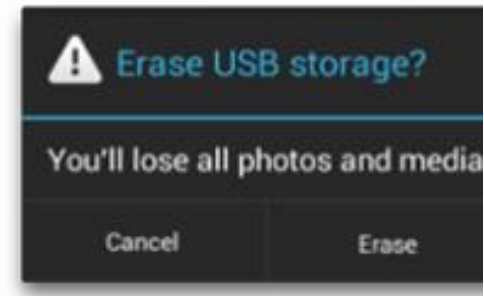
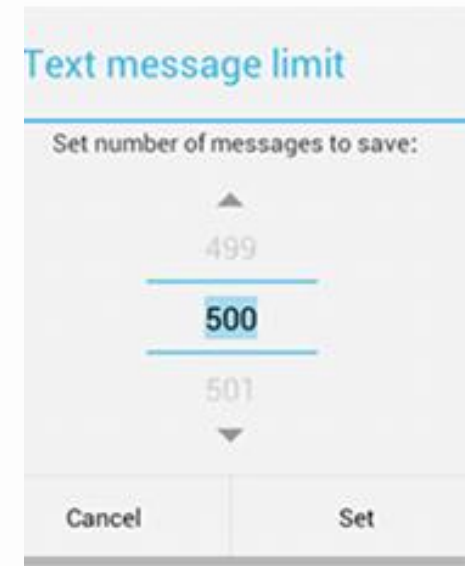
OPEN MIND - OPEN DOORS

Dialog

A dialog is a small window that prompts the user to make a decision or enter additional information. A dialog does not fill the screen and is normally used for modal events that require users to take an action before they can proceed.

The Dialog class is the base class for dialogs, but you should avoid instantiating Dialog directly. Instead, use one of the following subclasses:

- AlertDialog: A dialog that can show a title, up to three buttons, a list of selectable items, or a custom layout.
- DatePickerDialog or TimePickerDialog: A dialog with a pre-defined UI that allows the user to select a date or time.



Creating a Dialog Fragment

- » You can accomplish a wide variety of dialog designs—including custom layouts and those described in the Dialogs design guide—by extending DialogFragment and creating an AlertDialog in the onCreateDialog() callback method.
- » For example, here's a basic AlertDialog that's managed within a DialogFragment:

```
class StartGameDialogFragment : DialogFragment() {  
  
    override fun onCreateDialog(savedInstanceState: Bundle?): Dialog {  
        return activity?.let {  
            // Use the Builder class for convenient dialog construction  
            val builder = AlertDialog.Builder(it)  
            builder.setMessage(R.string.dialog_start_game)  
                .setPositiveButton(R.string.start,  
                    DialogInterface.OnClickListener { dialog, id ->  
                        // START THE GAME!  
                    })  
                .setNegativeButton(R.string.cancel,  
                    DialogInterface.OnClickListener { dialog, id ->  
                        // User cancelled the dialog  
                    })  
            // Create the AlertDialog object and return it  
            builder.create()  
        } ?: throw IllegalStateException("Activity cannot be null")  
    }  
}
```

Creating a Dialog Fragment (cont)

- » Now, when you create an instance of this class and call show() on that object, the dialog appears as shown in figure 1

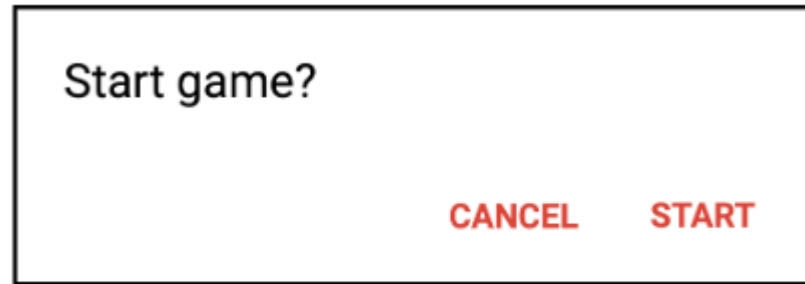


Figure 1. A dialog with a message and two action buttons.

Setting Screen

OPEN MIND - OPEN DOORS

About Settings

Settings allow users to change the functionality and behavior of an application. Settings can affect background behavior, such as how often the application synchronizes data with the cloud, or they can be more wide-reaching, such as changing the contents and presentation of the user interface.



Get started

A Preference is the basic building block of the Preference library. A settings screen contains a Preference hierarchy. You can define this hierarchy as an XML resource, or you can build a hierarchy in code.

The following sections describe how to build a simple settings screen using the AndroidX Preference library.

Before you start, add the Preference library dependency to your build.gradle file:

```
dependencies {  
    implementation("androidx.preference:preference-ktx:1.2.0")  
}
```

Create a hierarchy

In your project, navigate to res/xml folder, create a preferences.xml file, and add the following code to it:

```
<PreferenceScreen
    xmlns:app="http://schemas.android.com/apk/res-auto">

    <SwitchPreferenceCompat
        app:key="notifications"
        app:title="Enable message notifications"/>

    <Preference
        app:key="feedback"
        app:title="Send feedback"
        app:summary="Report technical issues or suggest new features"/>

</PreferenceScreen>
```

Inflate the hierarchy

To inflate a hierarchy from an XML attribute, create a **PreferenceFragmentCompat**, override **onCreatePreferences()**, and provide the XML resource to inflate, as shown in the following example:

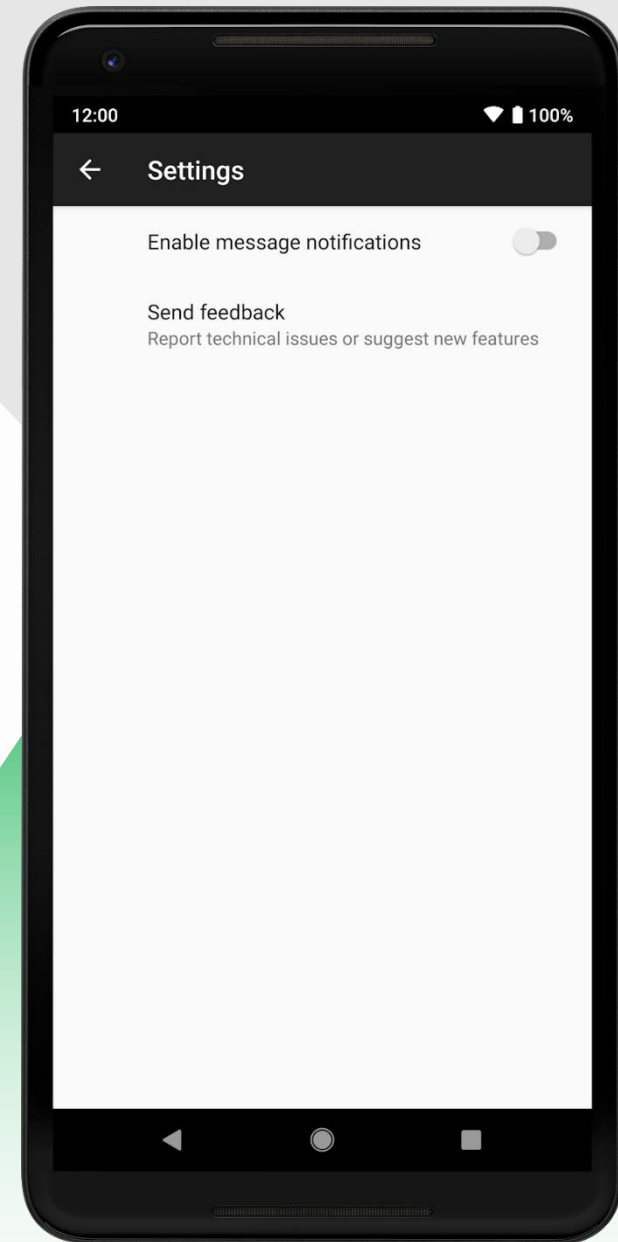
Kotlin Java

```
class MySettingsFragment : PreferenceFragmentCompat() {  
    override fun onCreatePreferences(savedInstanceState: Bundle?, rootKey: String?) {  
        setPreferencesFromResource(R.xml.preferences, rootKey)  
    }  
}
```

You can then add this Fragment to your Activity as you do with any other **Fragment**:

Kotlin Java

```
class MySettingsActivity : AppCompatActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        supportFragmentManager  
            .beginTransaction()  
            .replace(R.id.settings_container, MySettingsFragment())  
            .commit()  
    }  
}
```



Monitor the preferences

You can get an event when a preference changes by registering a listener for it:

Kotlin

Java

```
findPreference("notifications")
    ?.setOnPreferenceChangeListener { _, newValue ->
        Log.d("Preferences", "Notifications enabled: $newValue")
        true // Return true if the event is handled.
    }

findPreference("feedback")
    ?.setOnPreferenceClickListener {
        Log.d("Preferences", "Feedback was clicked")
        true // Return true if the click is handled.
    }
```

Read the current preference value

PreferenceFragmentCompat hides much of the machinery involved in saving and reading the preferences. However, everything is stored using **SharedPreferences**, and you can read these values as you normally do with **SharedPreferences**:

```
val preferences = PreferenceManager.getDefaultSharedPreferences(this).all

preferences.forEach {
    Log.d("Preferences", "${it.key} -> ${it.value}")
}
```

Reference

- [Organize your settings | Android Developers](#)
- [Customize your settings | Android Developers](#)
- [Use saved Preference values | Android Developers](#)
- [Create a hierarchy in code | Android Developers](#)
- [Handle other form factors | Android Developers](#)
- [Preference components and attributes | Android Developers](#)

ViewPager2

OPEN MIND - OPEN DOORS

Create swipe views with tabs using ViewPager2

Swipe views let you navigate between sibling screens, such as tabs, with a horizontal finger gesture, or swipe. This navigation pattern is also referred to as horizontal paging.

This topic teaches you how to create a tab layout with swipe views for switching between tabs and how to show a title strip instead of tabs.



Implement swipe views

You can create swipe views using AndroidX's **ViewPager2** widget. To use **ViewPager2** and tabs, you need to add a dependency on **ViewPager2** and Material Components to your project.

To set up your layout with **ViewPager2**, add the `<ViewPager2>` element to your XML layout. For example, if each page in the swipe view consumes the entire layout, then your layout looks like this:

```
<androidx.viewpager2.widget.ViewPager2
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/pager"
    android:layout_width="match_parent"
    android:layout_height="match_parent" />
```

Implement swipe views (cont)

To insert child views that represent each page, hook this layout to a **FragmentStateAdapter**. Here's how you might use it to swipe across a collection of **Fragment** objects:

```
class CollectionDemoFragment : Fragment() {
    // When requested, this adapter returns a DemoObjectFragment,
    // representing an object in the collection.
    private lateinit var demoCollectionAdapter: DemoCollectionAdapter
    private lateinit var viewPager: ViewPager2

    override fun onCreateView(
        inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        return inflater.inflate(R.layout.collection_demo, container, false)
    }

    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        demoCollectionAdapter = DemoCollectionAdapter(this)
        viewPager = view.findViewById(R.id.pager)
        viewPager.adapter = demoCollectionAdapter
    }
}

class DemoCollectionAdapter(fragment: Fragment) : FragmentStateAdapter(fragment) {

    override fun getItemCount(): Int = 100

    override fun createFragment(position: Int): Fragment {
        // Return a NEW fragment instance in createFragment(int).
        val fragment = DemoObjectFragment()
        fragment.arguments = Bundle().apply {
            // The object is just an integer.
            putInt(ARG_OBJECT, position + 1)
        }
        return fragment
    }
}

private const val ARG_OBJECT = "object"

// Instances of this class are fragments representing a single
// object in the collection.
class DemoObjectFragment : Fragment() {

    override fun onCreateView(
        inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View {
        return inflater.inflate(R.layout.fragment_collection_object, container, false)
    }

    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        arguments?.takeIf { it.containsKey(ARG_OBJECT) }?.apply {
            val textView: TextView = view.findViewById(android.R.id.text1)
            textView.text = getInt(ARG_OBJECT).toString()
        }
    }
}
```

Add tabs using a TabLayout

A TabLayout provides a way to display tabs horizontally. When used together with a ViewPager2, a TabLayout can provide a familiar interface for navigating between pages in a swipe view.

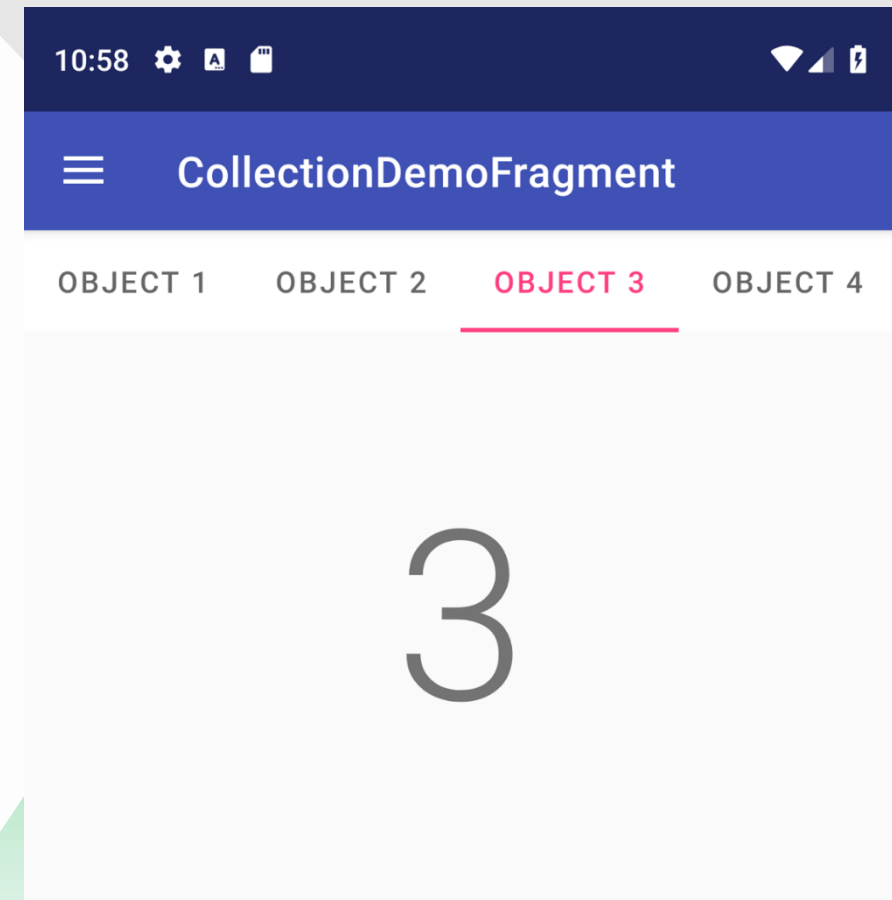
To include a TabLayout in a **ViewPager2**, add a **<TabLayout>** element above the **<ViewPager2>** element:

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical">

    <com.google.android.material.tabs.TabLayout
        android:id="@+id/tab_layout"
        android:layout_width="match_parent"
        android:layout_height="wrap_content" />

    <androidx.viewpager2.widget.ViewPager2
        android:id="@+id/pager"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1" />

</LinearLayout>
```



Add tabs using a TabLayout

Next, create a **TabLayoutMediator** to link the **TabLayout** to the **ViewPager2** and attach it, as follows:

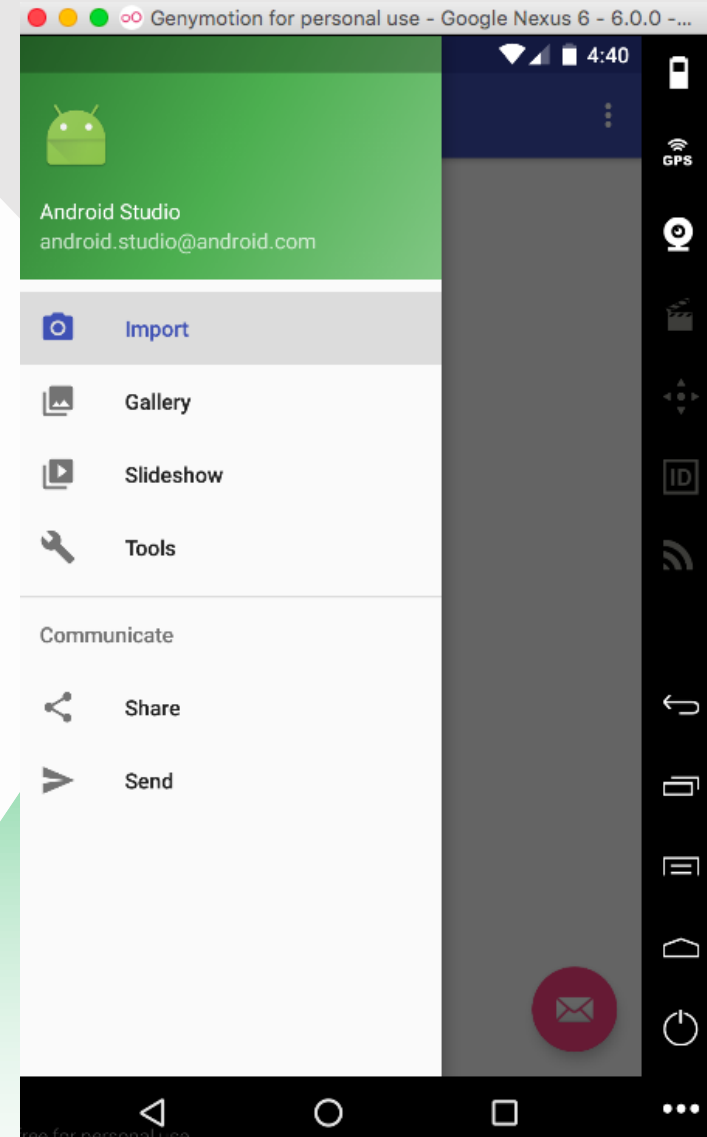
```
class CollectionDemoFragment : Fragment() {  
    ...  
    override fun onCreateView(view: View, savedInstanceState: Bundle?) {  
        val tabLayout = view.findViewById(R.id.tab_layout)  
        TabLayoutMediator(tabLayout, viewPager) { tab, position ->  
            tab.text = "OBJECT ${position + 1}"  
        }.attach()  
    }  
    ...  
}
```

Navigation View

OPEN MIND - OPEN DOORS

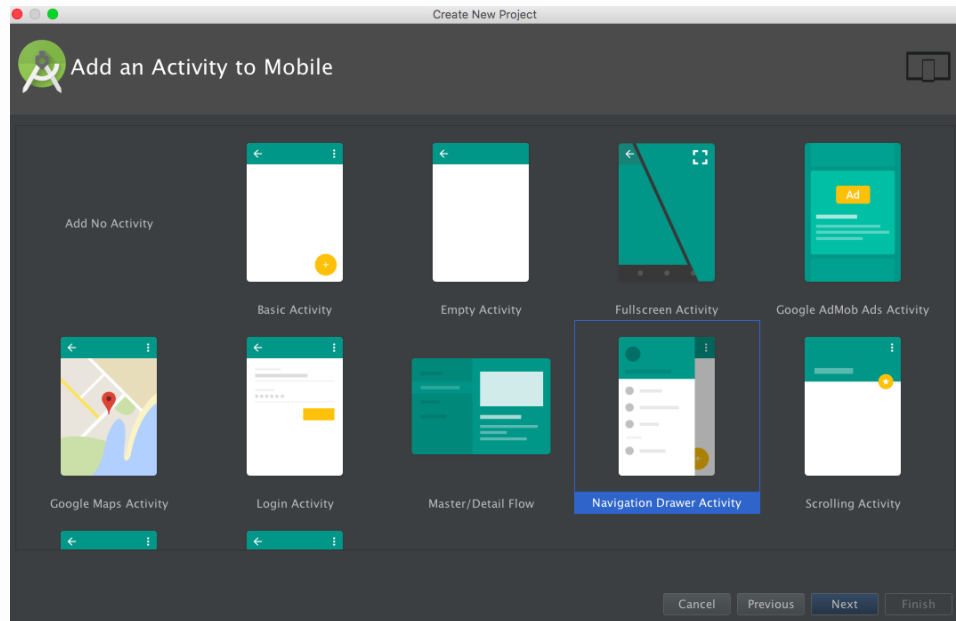
About

We have already implemented a Navigation Drawer in this tutorial and it was tiresome to code. NavigationView is a better and easier to implement alternative to a Navigation Drawer. NavigationDrawer required us to implement the items using a ListView/RecyclerView by implementing a custom Adapter. With the introduction of a NavigationView all we require is to inflate the items using menu resources that we'll see soon. NavigationView is generally placed inside a DrawerLayout.



NavigationView Getting Started

Android Studio provides us a ready made Navigation Drawer Activity that implements a standard Navigation Menu. You can choose it from the following dialog.



Understanding the NavigationView

The NavigationView class extends FrameLayout. It's defined in the xml under the tag as:

```
<android.support.design.widget.NavigationView/>
```

The NavigationView essentially consists of two major components:

- 1. HeaderView** : This View is typically displayed at the top of the Navigation Drawer. It essentially holds the profile picture, name email address and a background cover pic. This view is defined in a separate layout file that we'll look at in a bit. To add the layout into our NavigationView, the **app:headerLayout** parameter is used
- 2. Menu** : This is displayed below the HeaderView and it contains all the navigation items in the form of a list. The layout file for this is defined in the menus folder. To add the layout into the NavigationView , the **app:menu** parameter is used

Navigation Drawer

OPEN MIND - OPEN DOORS

About

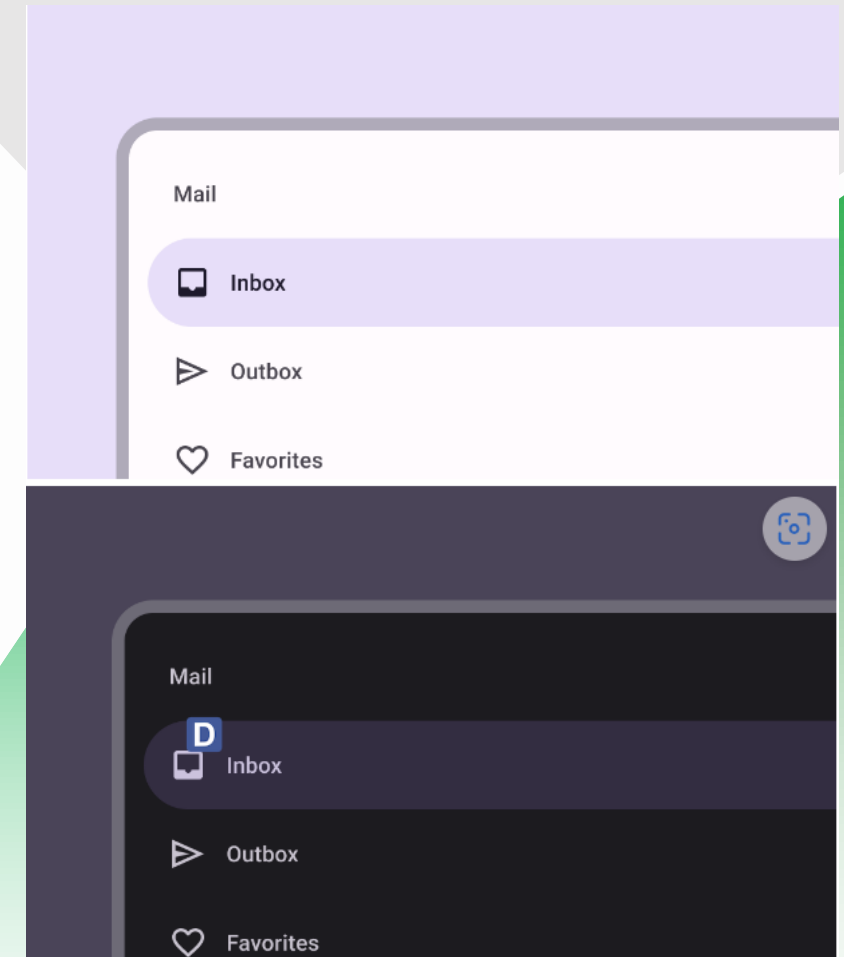
The **navigation drawer** component is a slide-in menu that lets users navigate to various sections of your app. Users can activate it by swiping from the side or tapping a menu icon.

Consider these three use cases for implementing a Navigation Drawer:

- **Content organization:** Enable users to switch between different categories, such as in news or blogging apps.
- **Account management:** Provide quick links to account settings and profile sections in apps with user accounts.
- **Feature discovery:** Organize multiple features and settings in a single menu to facilitate user discovery and access in complex apps.

In Material Design, there are two types of navigation drawers:

- **Standard:** Share space within a screen with other content.
- **Modal:** Appears over the top of other content within a screen.



Example

You can use the **ModalNavigationDrawer** composable to implement a navigation drawer.

Use the **drawerContent** slot to provide a **ModalDrawerSheet** and provide the drawer's contents, as in the following example:

```
ModalNavigationDrawer(  
    drawerContent = {  
        ModalDrawerSheet {  
            Text("Drawer title", modifier = Modifier.padding(16.dp))  
            Divider()  
            NavigationDrawerItem(  
                label = { Text(text = "Drawer Item") },  
                selected = false,  
                onClick = { /*TODO*/ }  
            )  
            // ...other drawer items  
        }  
    }  
) {  
    // Screen content  
}
```

Example(cont)

ModalNavigationDrawer accepts a number of additional drawer parameters. For example, you can toggle whether or not the drawer responds to drags with the **gesturesEnabled** parameter as in the following example:

```
ModalNavigationDrawer(  
    drawerContent = {  
        ModalDrawerSheet {  
            // Drawer contents  
        }  
    },  
    gesturesEnabled = false  
) {  
    // Screen content  
}
```

Control behavior

To control how the drawer opens and closes, use **DrawerState**. You should pass a **DrawerState** to **ModalNavigationDrawer** using the `drawerState` parameter.

DrawerState provides access to the open and close functions, as well as properties related to the current drawer state. These suspending functions require a `CoroutineScope`, which you can instantiate using **rememberCoroutineScope**. You can also call the suspending functions in response to UI events.

```
val drawerState = rememberDrawerState(initialValue = DrawerValue.Closed)
val scope = rememberCoroutineScope()
ModalNavigationDrawer(
    drawerState = drawerState,
    drawerContent = {
        ModalDrawerSheet { /* Drawer content */ }
    },
) {
    Scaffold(
        floatingActionButton = {
            ExtendedFloatingActionButton(
                text = { Text("Show drawer") },
                icon = { Icon(Icons.Filled.Add, contentDescription = "") },
                onClick = {
                    scope.launch {
                        drawerState.apply {
                            if (isClosed) open() else close()
                        }
                    }
                }
            )
        }
    ) {
        contentPadding ->
            // Screen content
    }
}
```

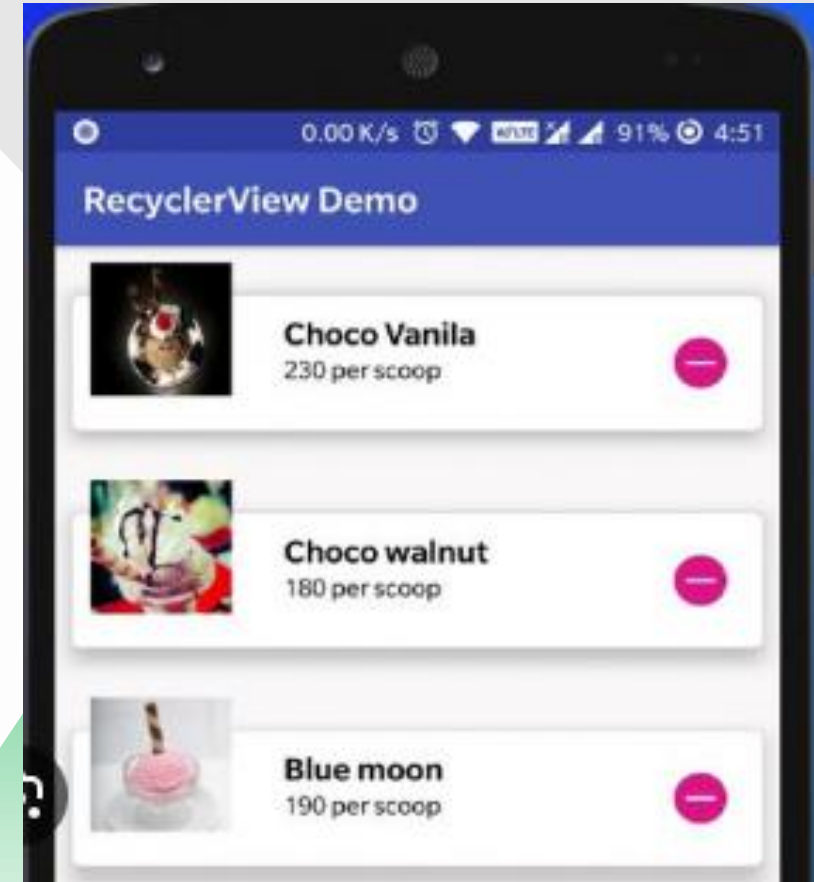
RecyclerView Adapter DiffUtil

OPEN MIND - OPEN DOORS

About

RecyclerView makes it easy to efficiently display large sets of data. You supply the data and define how each item looks, and the RecyclerView library dynamically creates the elements when they're needed.

As the name implies, RecyclerView recycles those individual elements. When an item scrolls off the screen, RecyclerView doesn't destroy its view. Instead, RecyclerView reuses the view for new items that have scrolled onscreen. RecyclerView improves performance and your app's responsiveness, and it reduces power consumption



Key classes

Several classes work together to build your dynamic list.

- **RecyclerView** is the **ViewGroup** that contains the views corresponding to your data. It's a view itself, so you add **RecyclerView** to your layout the way you would add any other UI element.
- Each individual element in the list is defined by a view holder object. When the view holder is created, it doesn't have any data associated with it. After the view holder is created, the **RecyclerView** binds it to its data. You define the view holder by extending **RecyclerView.ViewHolder**.
- The **RecyclerView** requests views, and binds the views to their data, by calling methods in the adapter. You define the adapter by extending **RecyclerView.Adapter**.
- The layout manager arranges the individual elements in your list. You can use one of the layout managers provided by the **RecyclerView** library, or you can define your own. Layout managers are all based on the library's **LayoutManager** abstract class.

Steps for implementing your RecyclerView

If you're going to use RecyclerView, there are a few things you need to do. They are explained in detail in the following sections.

1. Decide how the list or grid looks. Ordinarily, you can use one of the RecyclerView library's standard layout managers.
2. Design how each element in the list looks and behaves. Based on this design, extend the ViewHolder class. Your version of ViewHolder provides all the functionality for your list items. Your view holder is a wrapper around a View, and that view is managed by RecyclerView.
3. Define the Adapter that associates your data with the ViewHolder views.

Plan your layout

The items in your RecyclerView are arranged by a LayoutManager class. The RecyclerView library provides three layout managers, which handle the most common layout situations:

- **LinearLayoutManager** arranges the items in a one-dimensional list.
- **GridLayoutManager** arranges the items in a two-dimensional grid:
 - If the grid is arranged vertically, GridLayoutManager tries to make all the elements in each row have the same width and height, but different rows can have different heights.
 - If the grid is arranged horizontally, GridLayoutManager tries to make all the elements in each column have the same width and height, but different columns can have different widths.
- **StaggeredGridLayoutManager** is similar to **GridLayoutManager**, but it does not require that items in a row have the same height (for vertical grids) or items in the same column have the same width (for horizontal grids). The result is that the items in a row or column can end up offset from each other.

Implement your adapter and view holder

When you define your adapter, you override three key methods:

- **onCreateViewHolder()**: RecyclerView calls this method whenever it needs to create a new ViewHolder. The method creates and initializes the ViewHolder and its associated View, but does not fill in the view's contents—the ViewHolder has not yet been bound to specific data.
- **onBindViewHolder()**: RecyclerView calls this method to associate a ViewHolder with data. The method fetches the appropriate data and uses the data to fill in the view holder's layout. For example, if the RecyclerView displays a list of names, the method might find the appropriate name in the list and fill in the view holder's TextView widget.
- **getItemCount()**: RecyclerView calls this method to get the size of the dataset. For example, in an address book app, this might be the total number of addresses. RecyclerView uses this to determine when there are no more items that can be displayed.

```
class CustomAdapter(private val dataSet: Array<String>) :  
    RecyclerView.Adapter<CustomAdapter.ViewHolder>() {  
  
    /**  
     * Provide a reference to the type of views that you are using  
     * (custom ViewHolder)  
     */  
    class ViewHolder(view: View) : RecyclerView.ViewHolder(view) {  
        val textView: TextView  
  
        init {  
            // Define click listener for the ViewHolder's View  
            textView = view.findViewById(R.id.textView)  
        }  
    }  
  
    // Create new views (invoked by the layout manager)  
    override fun onCreateViewHolder(viewGroup: ViewGroup, viewType: Int): ViewHolder {  
        // Create a new view, which defines the UI of the list item  
        val view = LayoutInflater.from(viewGroup.context)  
            .inflate(R.layout.text_row_item, viewGroup, false)  
  
        return ViewHolder(view)  
    }  
  
    // Replace the contents of a view (invoked by the layout manager)  
    override fun onBindViewHolder(viewHolder: ViewHolder, position: Int) {  
  
        // Get element from your dataset at this position and replace the  
        // contents of the view with that element  
        viewHolder.textView.text = dataSet[position]  
    }  
  
    // Return the size of your dataset (invoked by the layout manager)  
    override fun getItemCount() = dataSet.size  
}
```

Implement your adapter and view holder

The layout for the each view item is defined in an XML layout file, as usual. In this case, the app has a text_row_item.xml file like this:

```
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="@dimen/list_item_height"
    android:layout_marginLeft="@dimen/margin_medium"
    android:layout_marginRight="@dimen/margin_medium"
    android:gravity="center_vertical">

    <TextView
        android:id="@+id/textView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/element_text"/>
</FrameLayout>
```

Next steps

```
class MainActivity : AppCompatActivity() {  
  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContentView(R.layout.activity_main)  
  
        val dataset = arrayOf("January", "February", "March")  
        val customAdapter = CustomAdapter(dataset)  
  
        val recyclerView: RecyclerView = findViewById(R.id.recycler_view)  
        recyclerView.adapter = customAdapter  
  
    }  
  
}
```

DiffUtil

It's a utility class that helps us to perform complex tasks easily, Eugene Myers' algorithm is the foundation of DiffUtils.

DiffUtils is used to track changes made to the RecyclerView Adapter.

DiffUtil notifies the RecyclerView of any changes to the data set using the following methods:

1. notifyItemMoved
2. notifyItemRangeChanged
3. notifyItemRangeInserted
4. notifyItemRangeRemoved

DiffUtil

These techniques are significantly more efficient than `notifyDataSetChanged()`. However, in order for `DiffUtils` to function in the project, we must give information about the old and new lists. `DiffUtil` is used for this. Request a callback. We'll make a class.

```
class CoursesCallback(private val oldList: List<Course>, private val newList: List<Course>) : DiffUtil.Callback() {
    override fun getCourseNew(): Int = oldList.size
    override fun getNewListSize(): Int = newList.size
    override fun areItemsTheSame(oldItemPosition: Int, newItemPosition: Int): Boolean {
        return oldList[oldItemPosition].courseNumber === newList.get(newItemPosition).rateIndex
    }
    override fun areContentsTheSame(oldCourse: Int, newPosition: Int): Boolean {
        val (_, value, name) = oldList[oldCourse]
        val (_, value1, name1) = newList[newPosition]
        return name == name1 && value == value1
    }
    @Nullable
    override fun geeksPayload(oldCourse: Int, newPosition: Int): Any? {
        return super.geeksPayload(oldCourse, newPosition)
    }
}
```


DiffUtil

- **getOldCourse():** This function returns the length of the old list.
- **getNewCourse():** Returns the new list's size.
- **areItemsTheSame(oldPosition:Int, newPosition:Int):** Called by the DiffUtil to determine whether two objects in the old and new lists represent the same Item.
- **areContentsTheSame(oldPosition:Int, newPosition:Int):** Determines if two objects have the same data. Depending on your UI, you may modify its behavior. DiffUtil calls this function only if areItemsTheSame returns true. In our example, we're contrasting the name and price of a certain item.
- **obtaingeeksPayload(oldPosition:Int, newPosition:Int):** If areItemTheSame returns true and areContentsTheSame returns false, the condition is satisfied. Diff This method is called by Util to obtain a payload about the modification

```
fun setCourses(newCourse: List<Courses>) {  
    val diffCallback = CoursesDiffCallback(ratings, newCourse)  
    val diffCourses = DiffUtil.calculateDiff(diffCallback)  
    courses.clear()  
    courses.addAll(newCourse)  
    diffCourses.dispatchUpdatesTo(this)  
}
```

What are the Benefits of using DiffUtils?

The performance chart below shows that using DiffUtil is superior in the case of RecyclerView. These findings are based on the Nexus 6P with M-

1. 100 items and 10 modifications: average: 0.39 milliseconds, median: 0.35 milliseconds
2. 3.82 ms for 100 items and 100 modifications, 3.75 ms for the median.
3. 2.09 ms for 100 items and 100 changes without movements, with a median of 2.06 ms.
4. 1000 items and 50 modifications: average: 4.67 milliseconds, median: 4.59 milliseconds
5. 1000 things and 50 changes with no moves: 3.59 ms on average, 3.50 ms on average
6. 1000 items and 200 modifications: 27.07 milliseconds, median: 26.92 milliseconds
7. 1000 items and 200 changes without moves: 13.54 milliseconds, median: 13.36 milliseconds



Thank You !

OPEN MIND - OPEN DOORS